Test report 3 – UI phase B: handles, snapping, parameter bar

Editing in the view · 2026-10-23 · Commit af6edd6

Phase B (Playwright)	Phase A (regression)	Vitest	Rust (changed crates)	Drag → coarse scene (cube / perforated plate)
14 / 14 passed	**22 / 22**	**128 / 128**	**33 / 33**	**36 ms / 371 ms**

1 Scope

Phase B adds editing in the 3D view to the browser UI. The code remains the single source of truth: handles change literals or write grid references, the parameter bar only changes the preview.

- **Handles** of the line under the cursor, only for a literal or a variable with a literal; computed values are shown as a locked handle (gray ring). Kinds: cube/square corner, box/rect corner points, radius (sphere, circle, cylinder, torus), height (cylinder, linear_extrude), cylinder between two points, translate target (arrow old → new, handle at the target), rotation ring per axis (rotate), scaling at the bounding box corner, grid planes with a literal (handle on an edge of the grid box, drag along the axis), point variables, polygon/spline points, zone center via translate. Colors: literal orange, default value of a module parameter violet, pure reference turquoise, locked gray.
- **Dragging:** `DragStart` → `Drag` (at most one message in flight, the last position wins) → `Edits` → the editor applies them as a live change and sends the source immediately; the engine tessellates coarsely, and finely after `DragEnd`. One drag is one undo step (live changes without history, at the end as one isolated event). Esc cancels.
- **Snapping** in screen space (snap radius configurable, default 14 px) to grid points of the module and visible anchors (`SnapTargets`), target highlighted and labeled; the reference is written. 1D handles (height, radius, plane) snap to grid planes of the same axis and write `g.name` (new in cad-eval). **Alt:** no snapping, numbers rounded to the step size (settings, default 1 mm / 15°).
- **Parameter bar** next to the view in module context: number (input field + slider), bool (switch), vector (three fields); reset per parameter, marking of changed values, "Reset all", values per module until the end of the session, "Apply as default" (`SetDefault`). Preview via `SetContext { overrides }`.
- **Inline controls** for number and bool literals of assignments at the end of the line (slider, switch); one slider drag is one undo step.
- **Ctrl+click** on a grid or anchor expression jumps to the declaration.

Changes to Rust crates: `cad-eval` (locked handles as `locked_handles`, `SnapRef::Plane` with a check that after writing, the handle lies exactly on the plane and the plane is visible at that location), `cad-session` (`Drag` always answers, `SetDragOptions`, scene with `locked_handles`), protocol types regenerated.

2 Test environment

Operating system	Linux 6.18.44-fc-v140 (x64)
CPU	Intel(R) Xeon(R) Processor @ 2.80GHz · 4 cores

Node.js	v22.22.0
Browser	Chromium 141.0.7390.37 (headless, Playwright 1.56.1)
Graphics	WebGL via SwiftShader (software rendering, no GPU)
Window	1440 × 900 px, device pixel ratio 1
Build	Vite 8.3.5 production build, preview server (vite preview)
Libraries	React 19.3.1, three.js 0.186.1, CodeMirror 6.43.15, Zustand 5.0.16, Vitest 5.0.5
Engine	cad_wasm_bg.wasm 1.17 MiB, smlib.wasm 4.85 MiB
Source	Branch worktree-agent-a590210bcbc87a815, commit af6edd6

The end-to-end tests run against the production build with real mouse events on the canvas (Playwright `mouse.down/move/up`, one step per animation frame). They wait for scene versions, the drag state and the editor text via the test hook `window.__cad`, never for fixed times. The hook only supplies screen positions (handles, grid points, points on a ring or axis).

3 Scenarios (Playwright, web/e2e/ui-b.spec.ts)

ID	Scenario	Result	Duration	Check
B01	Drag cube corner: literal live, scene follows, one undo	passed	7.6 s	Handle at the corner opposite the origin dragged with the mouse: the literal changed live (6 intermediate states with coarse tessellation), the scene followed. One Ctrl+Z restored "cube([20, 20, 20]);", Ctrl+Y (redo) restored "cube([22, 23, 22]);".
B02	Releasing the translate handle on a grid point writes g[...]	passed	13.7 s	translate handle (target of the arrow) dragged to the intersection point rechts/hinten/deckel: target within the snap radius highlighted and labeled; on release, the code contains the reference g[rechts, hinten, deckel] (no coordinates).
B03	Snapping to an anchor writes k.ecke	passed	13.1 s	Handle of the translate target dragged onto the diamond of the anchor ecke of the named instance k: label "k.ecke", on release k.ecke is written.
B04	Alt-drag: free, numbers rounded to the step size	passed	17.2 s	Dragged close to a grid point with the Alt key held: no snapping, numbers are written, rounded to 1 mm (translate([39, 31, 1]) cube(4);). Then step size set to 0.5 mm in the settings: translate([37, 28.5, -0.5]) cube(4);.
B05	Drag a grid plane with a literal	passed	5.9 s	Cursor in the grid block: all planes with a literal as a solid line with a handle. The handle of the plane rechts was dragged along the x axis; the literal follows, the line and grid points move along.
B06	Height snaps to a grid plane of the same axis (g.deckel)	passed	6.5 s	Height handle of the cylinder dragged to the grid plane deckel: planes of the same axis are snap targets, g.deckel is written (new in cad-eval: SnapRef::Plane).
B07	rotate ring snaps in 15° steps	passed	7.8 s	Rotation rings per axis; the handle on the z ring was dragged from 10° to 52°. Only multiples of 15° are written (step size for angles), 45° at the end.
B08	Locked handle for a computed value	passed	2.9 s	Computed value [a + 1, 0, 0] * 4: the handle is shown (gray ring) but locked. Dragging changes nothing, the status bar reports "computed value – locked" (German UI: „berechneter Wert – gesperrt“).

B09	Variable with a literal: definition changes, all uses move	passed	8.8 s	translate(p) in line 2 dragged: the definition of p in line 1 changes, cube and sphere (lines 2 and 3) move by the same amount.
B10	Zone center via translate snaps to the grid	passed	12.3 s	Cursor on the zone (sphere) in the fillet block: handle at the zone center (translate), dragged to the corner rechts/hinten/deckel. The zone moves, the selected edges follow.
B11	Parameter bar: preview, reset, apply as default	passed	5.0 s	Cursor in module gehaeuse: bar with w, d, h, t (input field + slider, reset ↺). w = 60 changed only the preview (code unchanged, line marked), reset restored 40; h set to 31 via the slider. d = 50 was kept when leaving and re-entering the module; "Apply as default" (German UI: „Als Standard übernehmen“) wrote d = 50 into the module definition.
B12	Inline slider and switch	passed	3.7 s	Number literal of an assignment: small slider at the end of the line, dragged with the mouse from 3 to 7 (code and scene follow live, one undo step). Bool literal: switch, a click writes false; the scene switches to the else branch.
B13	Ctrl+click on a grid expression jumps to the grid declaration	passed	1.5 s	Ctrl+click on g[links, vorne, boden] (line 11) sets the cursor on the declaration grid g (line 4); a normal click still only sets the cursor.
B14	Measurements: latency drag → scene (coarse/fine), fps while dragging	passed	25.5 s	lochplatte (difference with fillet = 1, four holes): plane oben = t (default value, different color) moved 78 times, one step per frame. Drag → coarse scene: median 371 ms, p95 738 ms (15 scenes); release → fine scene 202 ms; 21 fps while dragging, 76 drag messages, 0 skipped.

Regression: the 22 scenarios from phase A (web/e2e/ui.spec.ts) ran in the same run, 22 passed.

4 Unit tests

Vitest file	passed	Failed
web/src/editor/decorations.test.ts	7 / 7	–
web/src/editor/editGroup.test.ts new	7 / 7	–
web/src/editor/lang.test.ts	42 / 42	–
web/src/logic/context.test.ts	5 / 5	–
web/src/logic/display.test.ts	15 / 15	–
web/src/logic/drag.test.ts new	9 / 9	–
web/src/logic/format.test.ts new	5 / 5	–
web/src/logic/grids.test.ts	5 / 5	–
web/src/logic/handles.test.ts new	12 / 12	–
web/src/logic/params.test.ts new	7 / 7	–
web/src/logic/positions.test.ts	5 / 5	–
web/src/logic/sceneIndex.test.ts	9 / 9	–

New: snap search in screen space, ray → value (plane, axis, ring), preview rounding, "at most one drag in flight", rounding and number formatting, undo grouping with the real CodeMirror history, mapping of the engine edits, state of the parameter bar, inline controls. Fixture `griffe.scad` from the real engine.

Rust test program	passed	new tests
tests/handles.rs	24 / 24	`computed_values_have_locked_handles`, `plane_snap_respects_visibility`, `one_d_handles_snap_to_grid_planes`, `zone_center_handle_through_translate`
tests/protocol_ts.rs	1 / 1	–
tests/session.rs	8 / 8	`drag_always_answers_and_plane_snap`

5 Measurements

Quantity	Value	Measurement
Drag → scene, coarse (median)	371 ms	Drag message until the scene of the source written from it is drawn, coarse tessellation (10 × tolerance); 15 scenes
Drag → scene, coarse (p95 / max)	738 ms / 738 ms	95 % quantile and maximum of the same measurement
Release → fine scene	202 ms	pointerup until the scene with the fine root mesh is drawn (DragEnd, closing the undo group, re-evaluation from the cache)
Frame rate while dragging	21 fps	requestAnimationFrame counter while dragging (19.9 s); longest gap 349 ms
Mouse moves / drag messages	78 / 76	at most one message in flight, 0 intermediate states skipped (last position wins)
Light model: drag → coarse scene (median / p95)	36 ms / 172 ms	B01, cube([20, 20, 20]): 6 scenes; cost of UI, worker and protocol without significant kernel work
Light model: release → fine / fps	23 ms / 35 fps	longest frame gap 412 ms, 15 drag messages
Largest image file in the report	57 KiB	JPEG, quality 82

Upper part: spec/lochplatte.scad, module lochplatte, plane oben = t (default value). Each change of t fully re-evaluates the difference with fillet and four holes (no cache hit); the latency here is determined by the kernel time (SetSource → complete scene median 174 ms). Equal rounded values produce no change; each change waits for its scene before the next position is sent (otherwise every new source would cancel the running kernel job). The frame rate is limited by software rendering (SwiftShader); phase A measured 14 fps while rotating.

6 Known limitations and deviations

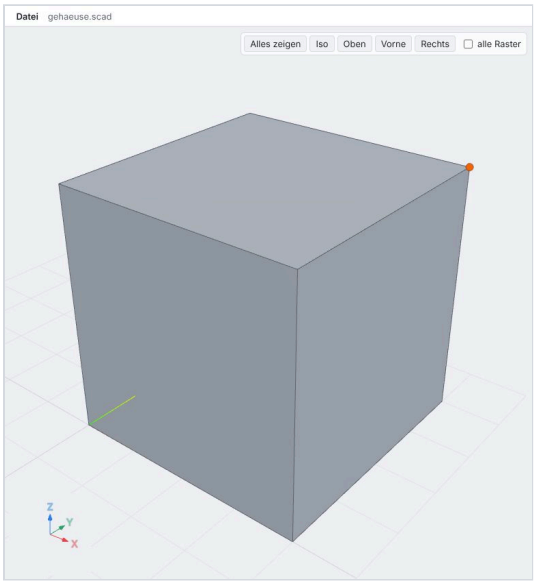
- **Jump to the grid declaration with Ctrl+click** instead of a plain click: a plain click must still set the cursor, otherwise grid expressions would not be editable. The UI only knows the references of a module in module context (cursor in the module).
- **Without Alt**, the handle snaps if a target is within the snap radius, otherwise numbers are written (rounded). A handle that consists only of references (e.g. `box(g[…], g[…])`) never turns into numbers without Alt; it only switches between targets.
- **Snapping of 1D handles** only to planes on which the handle lands exactly after writing `g.name` (same origin and scale along the axis). Under a translated subtree or with `center = true`/`d =` (factor 2) there are therefore no plane targets; otherwise an expression such as `g.deckel - 5` would be written, which locks the handle.
- Point handles are dragged in the plane through the handle perpendicular to the view direction (2D points in their xy plane); nothing changes along the view direction. For a specific axis, a standard view helps.

- Undo group: at the end of the drag the total change is undone once and re-applied as one history entry; this causes one additional re-evaluation from the cache.
- Inline controls only appear for assignments with a number or bool literal (spec, value types); the range depends on the value at the start of the drag (0 … 2 × value, at least 10).
- Parameters without a default value are set to 0 / false / [0, 0, 0] on reset (spec table); strings and lists are read-only.
- The measurements come from headless Chromium with SwiftShader.

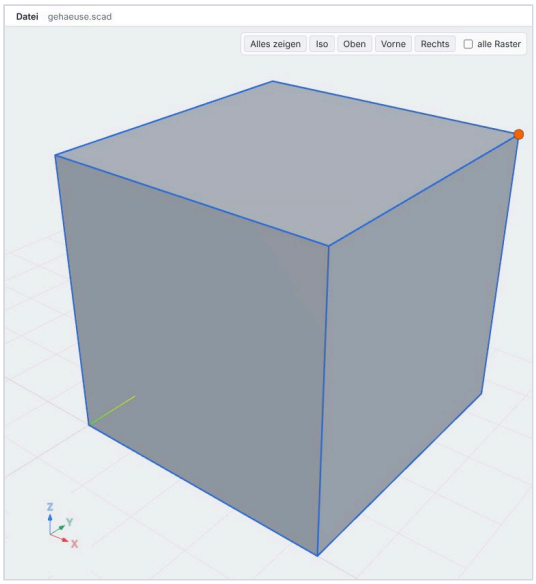
7 Screenshots and code excerpts

B01 Drag cube corner

BEFORE



AFTER



CODE BEFORE

```
cube([20, 20, 20]);
```

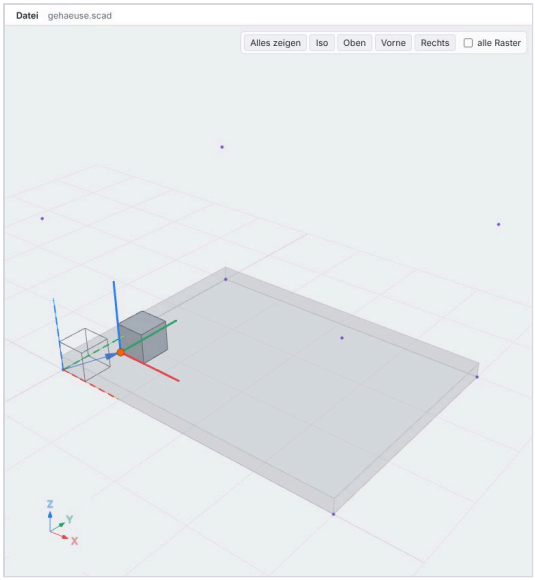
CODE AFTER

```
cube([22, 23, 22]);
```

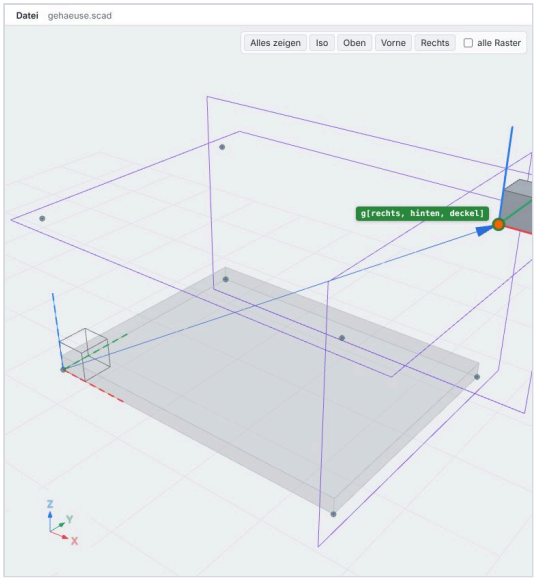
Handle at the corner opposite the origin dragged with the mouse: the literal changed live (6 intermediate states with coarse tessellation), the scene followed. One Ctrl+Z restored "cube([20, 20, 20]);", Ctrl+Y (redo) restored "cube([22, 23, 22]);".

B02 Snap to grid point

BEFORE



DURING



CODE BEFORE

```
translate([5, 5, 2]) cube(4);
```

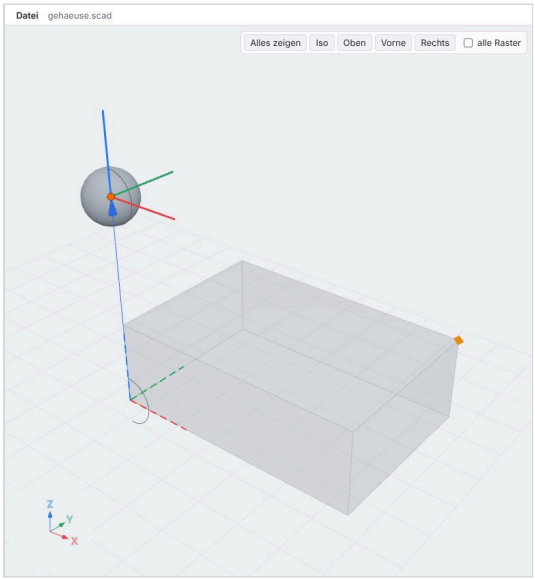
CODE AFTER

```
translate(g[rechts, hinten, deckel]) cube(4);
```

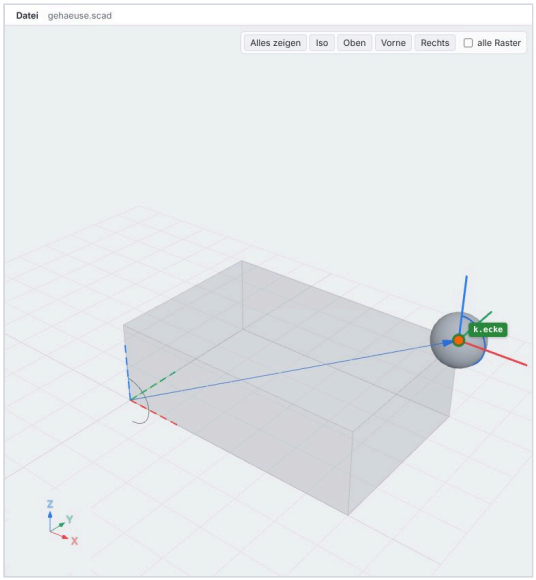
translate handle (target of the arrow) dragged to the intersection point rechts/hinten/deckel: target within the snap radius highlighted and labeled; on release, the code contains the reference g[rechts, hinten, deckel] (no coordinates).

B03 Snap to anchor

BEFORE



DURING



CODE BEFORE

```
translate([0, 0, 25]) sphere(3);
```

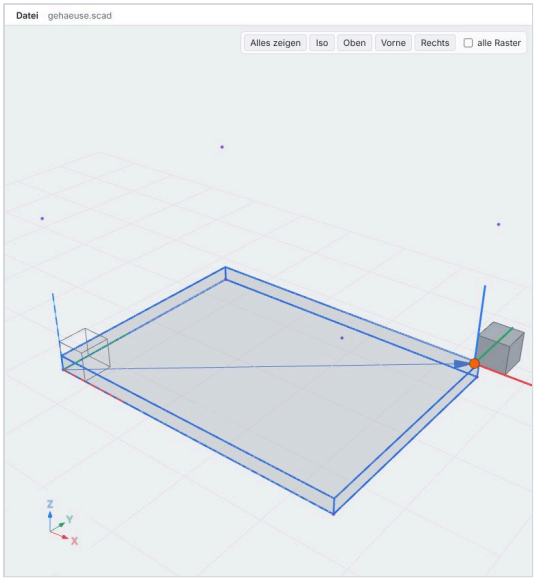
CODE AFTER

```
translate(k.ecke) sphere(3);
```

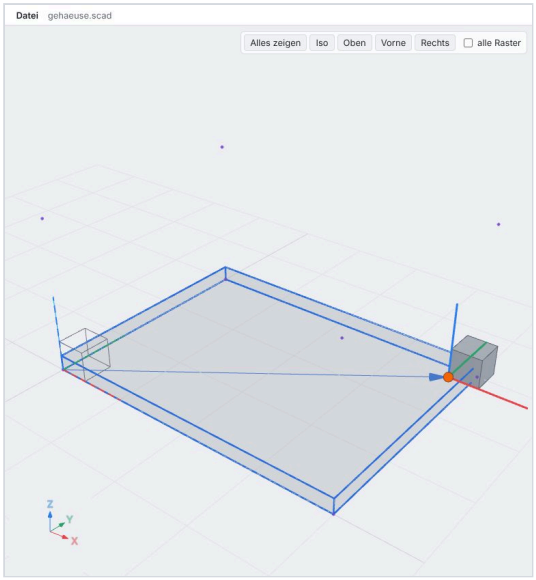
Handle of the translate target dragged onto the diamond of the anchor ecke of the named instance k: label "k.ecke", on release k.ecke is written.

B04 Alt-drag

DURING



AFTER



CODE BEFORE

```
translate([5, 5, 2]) cube(4);
```

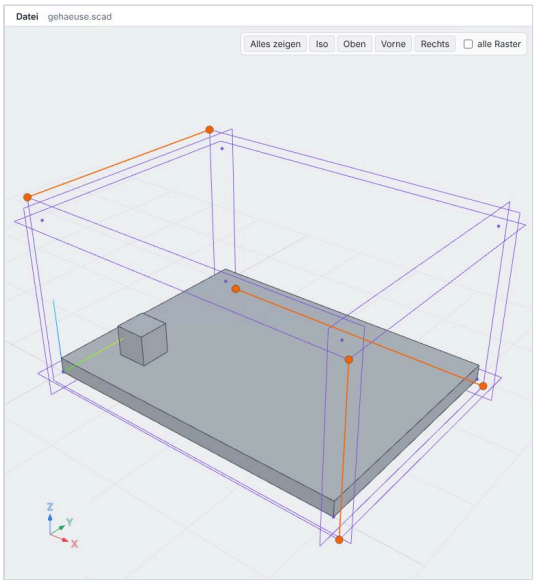
CODE AFTER

```
translate([39, 31, 1]) cube(4);
translate([37, 28.5, -0.5]) cube(4);
```

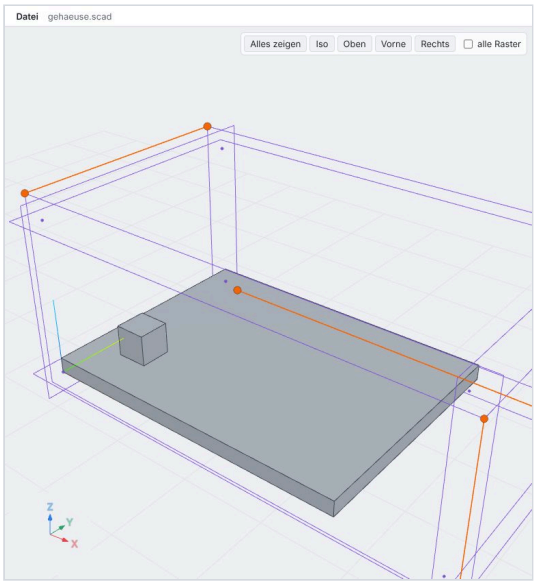
Dragged close to a grid point with the Alt key held: no snapping, numbers are written, rounded to 1 mm (translate([39, 31, 1]) cube(4);). Then step size set to 0.5 mm in the settings: translate([37, 28.5, -0.5]) cube(4);.

B05 Drag grid plane

BEFORE



AFTER



CODE BEFORE

```
    x: links = 0, rechts = 40;
```

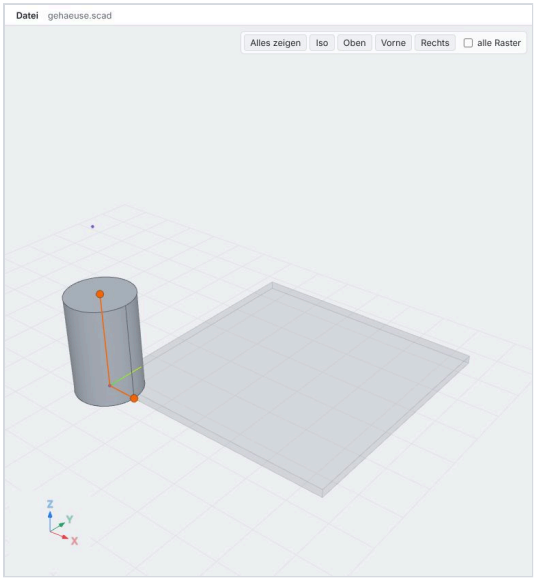
CODE AFTER

```
    x: links = 0, rechts = 52;
```

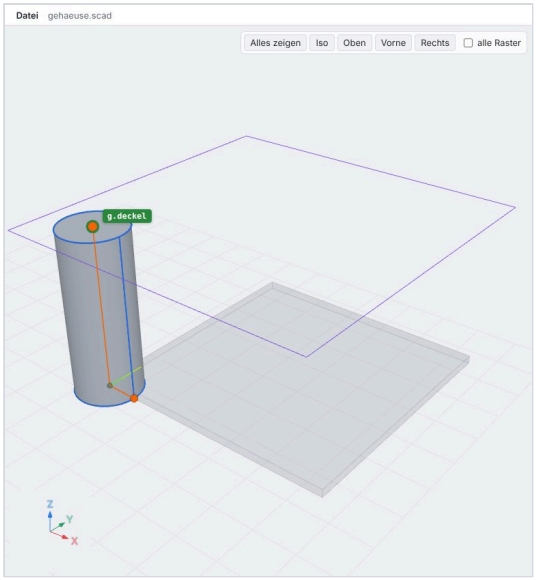
Cursor in the grid block: all planes with a literal as a solid line with a handle. The handle of the plane rechts was dragged along the x axis; the literal follows, the line and grid points move along.

B06 1D handle snaps to plane

BEFORE



DURING



CODE BEFORE

```
cylinder(h = 12, r = 4);
```

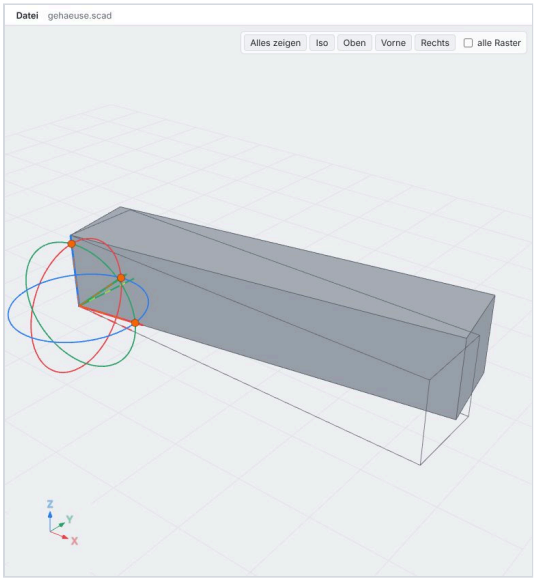
CODE AFTER

```
cylinder(h = g.deckel, r = 4);
```

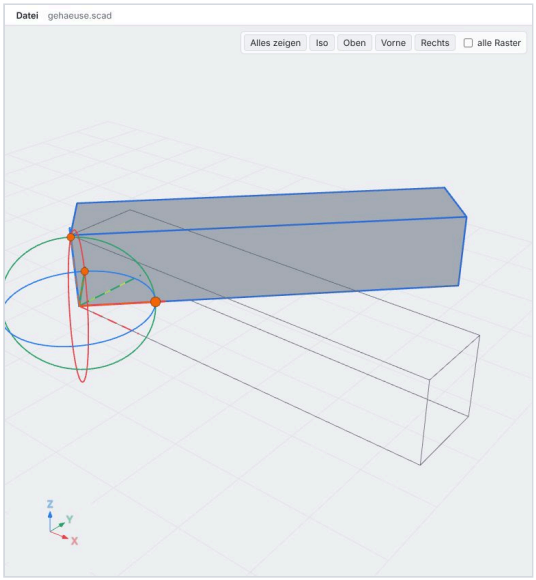
Height handle of the cylinder dragged to the grid plane deckel: planes of the same axis are snap targets, g.deckel is written (new in cad-eval: SnapRef::Plane).

B07 Rotation ring snaps in 15° steps

BEFORE



AFTER



CODE BEFORE

```
rotate([0, 0, 10]) cube([30, 6, 6]);
```

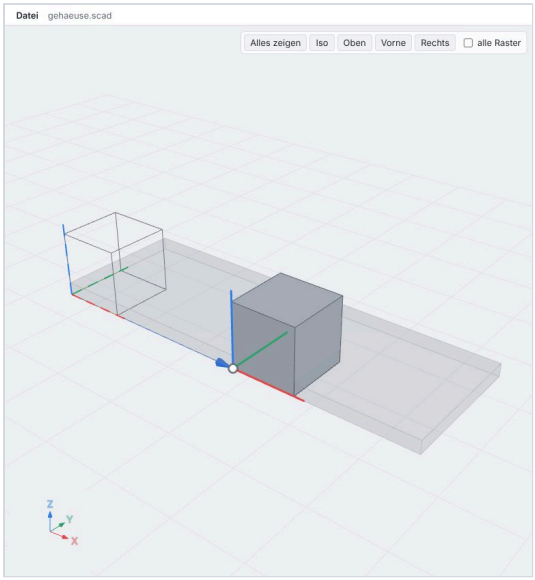
CODE AFTER

```
rotate([0, 0, 45]) cube([30, 6, 6]);
```

Rotation rings per axis; the handle on the z ring was dragged from 10° to 52°. Only multiples of 15° are written (step size for angles), 45° at the end.

B08 Locked handle

BEFORE



CODE BEFORE

```
translate([a + 1, 0, 0] * 4) cube(5);
```

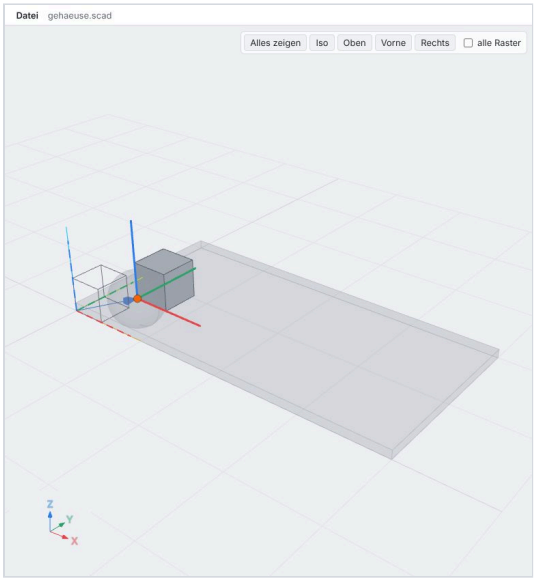
CODE AFTER

```
translate([a + 1, 0, 0] * 4) cube(5);
```

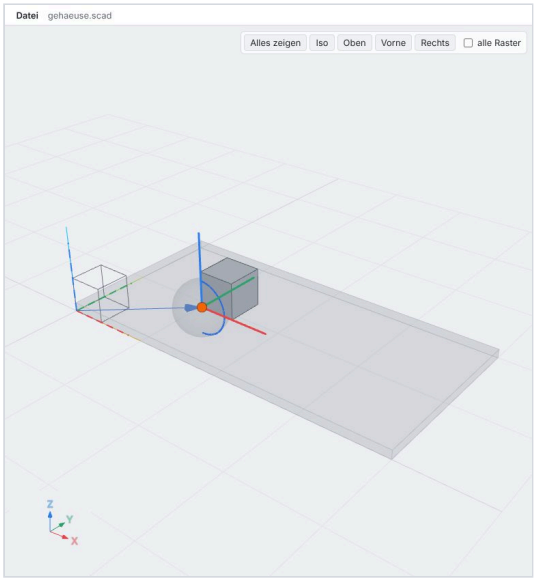
Computed value [a + 1, 0, 0] * 4: the handle is shown (gray ring) but locked. Dragging changes nothing, the status bar reports "computed value – locked".

B09 Variable with a literal

BEFORE



AFTER



CODE BEFORE

```
p = [10, 0, 5];
translate(p) cube(4);
translate(p) sphere(3);
```

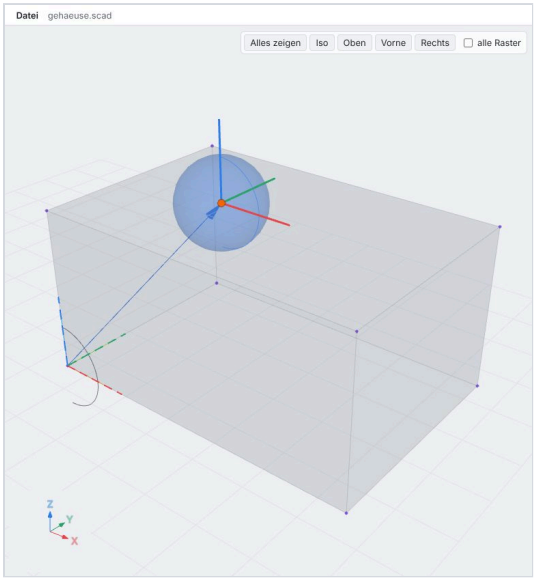
CODE AFTER

```
p = [15, 4, 4];
translate(p) cube(4);
translate(p) sphere(3);
```

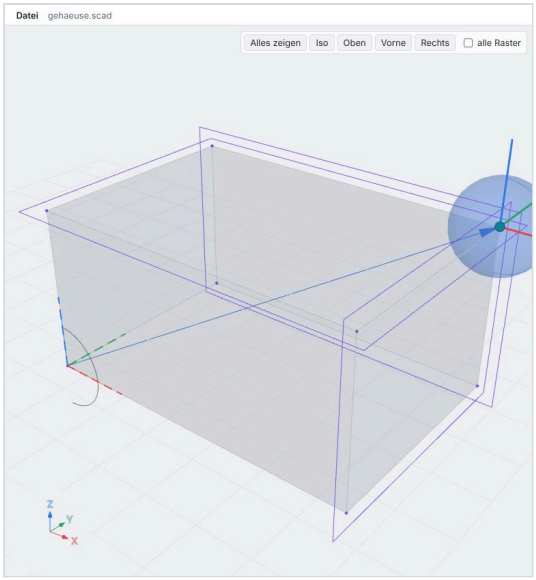
translate(p) in line 2 dragged: the definition of p in line 1 changes, cube and sphere (lines 2 and 3) move by the same amount.

B10 Zone center

BEFORE



AFTER



CODE BEFORE

```
    translate([10, 10, 15]) sphere(4);
```

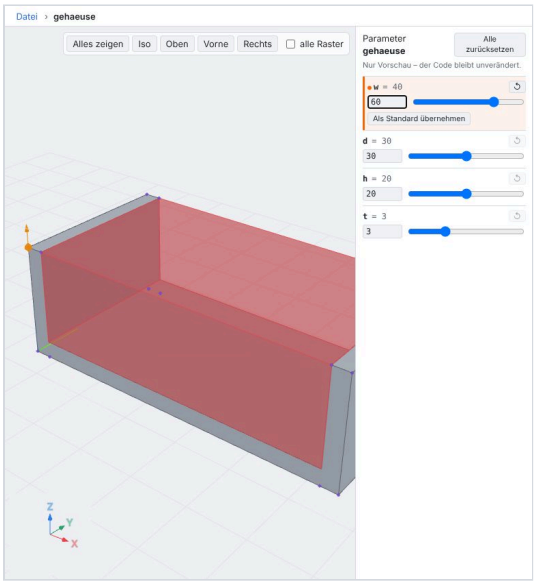
CODE AFTER

```
    translate(g[rechts, hinten, deckel])
sphere(4);
```

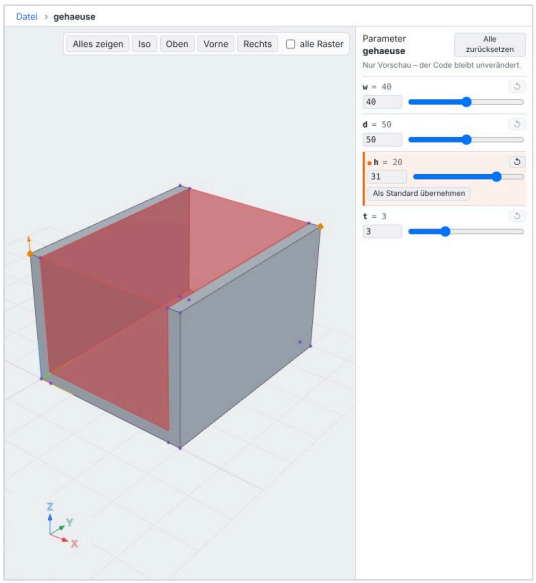
Cursor on the zone (sphere) in the fillet block: handle at the zone center (translate), dragged to the corner rechts/hinten/deckel. The zone moves, the selected edges follow.

B11 Parameter bar

BEFORE



AFTER



CODE BEFORE

```
module gehaeuse(w = 40, d = 30, h = 20, t = 3) {
```

CODE AFTER

```
module gehaeuse(w = 40, d = 50, h = 20, t = 3) {
```

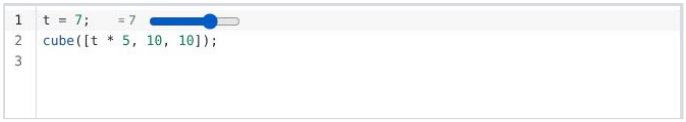
Cursor in module gehaeuse: bar with w, d, h, t (input field + slider, reset ↺). w = 60 changed only the preview (code unchanged, line marked), reset restored 40; h set to 31 via the slider. d = 50 was kept when leaving and re-entering the module; "Apply as default" wrote d = 50 into the module definition.

B12 Inline controls

BEFORE



AFTER



CODE BEFORE

```
t = 3;
hohl = true;
```

CODE AFTER

```
t = 7;
hohl = false;
```

Number literal of an assignment: small slider at the end of the line, dragged with the mouse from 3 to 7 (code and scene follow live, one undo step). Bool literal: switch, a click writes false; the scene switches to the else branch.

B13 Jump to grid declaration

CODE BEFORE

```
11: box(g[links, vorne, boden], g[rechts, hinten,
deckel]);
```

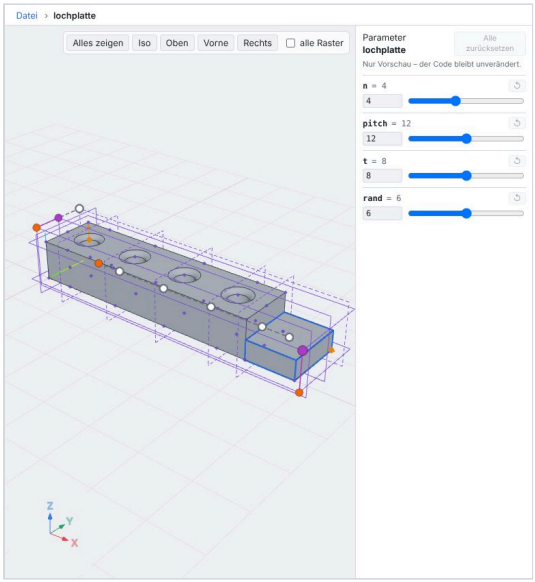
CODE AFTER

```
4: grid g {   ← Cursor
```

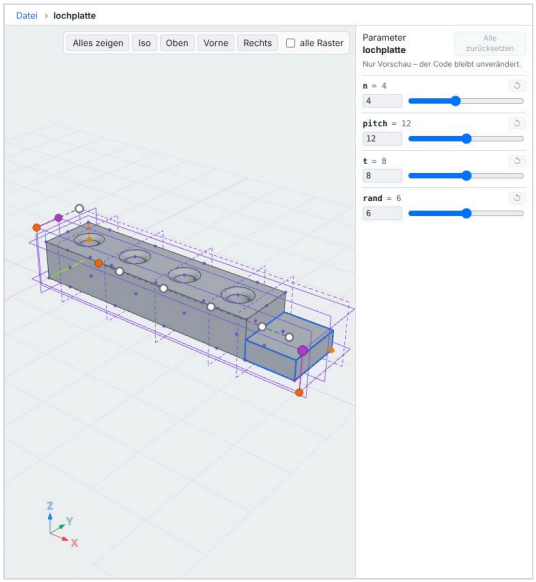
Ctrl+click on g[links, vorne, boden] (line 11) sets the cursor on the declaration grid g (line 4); a normal click still only sets the cursor.

B14 Measurements while dragging

DURING



AFTER



CODE BEFORE

```
module lochplatte(n = 4, pitch = 12, t = 4, rand =
6) {
```

CODE AFTER

```
module lochplatte(n = 4, pitch = 12, t = 8, rand =
6) {
```

lochplatte (difference with fillet = 1, four holes): plane oben = t (default value, different color) moved 78 times, one step per frame. Drag → coarse scene: median 371 ms, p95 738 ms (15 scenes); release → fine scene 202 ms; 21 fps while dragging, 76 drag messages, 0 skipped.